PYTHON ACTIVITY-BASED LEARNING

Case Study Solutions

Student Name	Jayaram
Roll Number	26001036
Subject	Python Programming

Selected Case Studies 1–10

Each case includes a problem statement, objectives, assumptions, algorithm, flowchart, pseudocode, Python code, sample expected result, and conclusion. Values and rules not supplied in the question sheet are illustrative assumptions; replace them if your faculty has given different rules.

Scanned with OKEN Scanner

1. University Scholarship Allocation

Problem Statement

Check marks, attendance, and family income; rank eligible applicants.

Objectives

Apply conditions and calculations correctly, validate inputs where relevant, and display a clear result.

Assumptions

Assume 3 subjects; each >= 40, average >= 75, attendance >= 75%, income <= Rs. 250000. Ties are decided by attendance.

Algorithm

1. Start. 2. Read the required inputs. 3. Validate inputs. 4. Calculate totals and apply the specified conditions. 5. Display the result. 6. Stop.

Flowchart (text form)

START → Input → Validate → Calculate / Apply Conditions → Display Result → STOP

Pseudocode

```
START\nINPUT required values\nVALIDATE input values\nCALCULATE totals and apply conditions\nDISPLAY result\nSTOP
```

Python Source Code

```
applicants = [
    ("Asha", [88, 81, 90], 92, 180000),
    ("Ravi", [78, 80, 76], 85, 220000),
    ("Meena", [70, 72, 68], 90, 200000),
    ("Kiran", [91, 89, 94], 70, 150000)
]
eligible = []
for name, marks, attendance, income in applicants:
    avg = sum(marks) / len(marks)
    if min(marks) >= 40 and avg >= 75 and attendance >= 75 and income <= 250000:
        eligible.append((name, avg, attendance))
eligible.sort(key=lambda x: (x[1], x[2]), reverse=True)
for rank, student in enumerate(eligible, 1):
    print(rank, student)
```

Sample Test Case / Expected Result

Expected: Asha and Ravi qualify; students are ranked by average marks, then attendance.

Conclusion

The program uses beginner-level Python variables, arithmetic, conditions, loops, and lists to solve the case study under the stated assumptions.

Scanned with OKEN Scanner

2. Progressive Electricity Billing

Problem Statement

Calculate monthly electricity bills using progressive slabs and flag a large increase.

Objectives

Apply conditions and calculations correctly, validate inputs where relevant, and display a clear result.

Assumptions

Assume first 100 units Rs. 2/unit, next 100 Rs. 3/unit, above 200 Rs. 5/unit; fixed charge Rs. 50; surcharge 5%; Rs. 20 rebate for <= 100 units. Increase above 30% is unusual.

Algorithm

1. Start. 2. Read the required inputs. 3. Validate inputs. 4. Calculate totals and apply the specified conditions. 5. Display the result. 6. Stop.

Flowchart (text form)

START → Input → Validate → Calculate / Apply Conditions → Display Result → STOP

Pseudocode

```
START\nINPUT required values\nVALIDATE input values\nCALCULATE totals and apply conditions\nDISPLAY result\nSTOP
```

Python Source Code

```
def energy_charge(units):
    charge = min(units, 100) * 2
    if units > 100:
        charge += min(units - 100, 100) * 3
    if units > 200:
        charge += (units - 200) * 5
    return charge

previous = None
for month, units in enumerate([90, 110, 260], 1):
    energy = energy_charge(units)
    bill = energy + 50 + energy * 0.05
    if units <= 100:
        bill -= 20
    print("Month", month, "bill Rs.", round(bill, 2))
    if previous is not None and (bill - previous) / previous > 0.30:
        print("Unusual increase detected")
    previous = bill
```

Sample Test Case / Expected Result

Expected: Month 1 bill Rs. 219.00. A substantial rise in later bills is flagged.

Conclusion

The program uses beginner-level Python variables, arithmetic, conditions, loops, and lists to solve the case study under the stated assumptions.

Scanned with OKEN Scanner

3. Hospital Billing and Patient Classification

Problem Statement

Calculate hospital charges, apply an age concession, validate amounts, and classify payment.

Objectives

Apply conditions and calculations correctly, validate inputs where relevant, and display a clear result.

Assumptions

Assume patients aged 60+ get 10% off consultation and treatment charges. Diagnostic charges are not discounted. Negative charges are invalid.

Algorithm

1. Start. 2. Read the required inputs. 3. Validate inputs. 4. Calculate totals and apply the specified conditions. 5. Display the result. 6. Stop.

Flowchart (text form)

START → Input → Validate → Calculate / Apply Conditions → Display Result → STOP

Pseudocode

```
START\nINPUT required values\nVALIDATE input values\nCALCULATE totals and apply conditions\nDISPLAY result\nSTOP
```

Python Source Code

```
age = 65
consultation = 500
diagnostics = 1200
treatment = 4000
paid = 4000

if min(consultation, diagnostics, treatment, paid) < 0:
    print("Invalid billing data")
else:
    concession = 0.10 * (consultation + treatment) if age >= 60 else 0
    total = consultation + diagnostics + treatment - concession
    balance = total - paid
    status = "Paid" if balance <= 0 else "Pending"
    print("Final bill: Rs.", total)
    print("Balance: Rs.", max(0, balance))
    print("Status:", status)
```

Sample Test Case / Expected Result

Expected: Final bill Rs. 5250; balance Rs. 1250; status Pending.

Conclusion

The program uses beginner-level Python variables, arithmetic, conditions, loops, and lists to solve the case study under the stated assumptions.

Scanned with OKEN Scanner

4. Employee Payroll and Deduction Analysis

Problem Statement

Calculate gross salary, progressive deductions, and net salary.

Objectives

Apply conditions and calculations correctly, validate inputs where relevant, and display a clear result.

Assumptions

Assume overtime = hours x hourly rate. Deduction: 0% on first Rs. 10000, 5% on next Rs. 10000, and 10% above Rs. 20000.

Algorithm

1. Start. 2. Read the required inputs. 3. Validate inputs. 4. Calculate totals and apply the specified conditions. 5. Display the result. 6. Stop.

Flowchart (text form)

START → Input → Validate → Calculate / Apply Conditions → Display Result → STOP

Pseudocode

```
START\nINPUT required values\nVALIDATE input values\nCALCULATE totals and apply conditions\nDISPLAY result\nSTOP
```

Python Source Code

```
basic = 20000
allowances = 5000
overtime_hours = 10
hourly_rate = 200
incentive = 1500
gross = basic + allowances + overtime_hours * hourly_rate + incentive
deduction = 0
if gross > 10000:
    deduction += min(gross - 10000, 10000) * 0.05
if gross > 20000:
    deduction += (gross - 20000) * 0.10
net = gross - deduction
print("Gross salary: Rs.", gross)
print("Deduction: Rs.", deduction)
print("Net salary: Rs.", net)
```

Sample Test Case / Expected Result

Expected: Gross Rs. 28500; deduction Rs. 1850; net salary Rs. 26650.

Conclusion

The program uses beginner-level Python variables, arithmetic, conditions, loops, and lists to solve the case study under the stated assumptions.

Scanned with OKEN Scanner

5. Multi-Plan Telecommunications Cost Optimization

Problem Statement

Compare plans for usage, validity, and budget, including extra-data charges.

Objectives

Apply conditions and calculations correctly, validate inputs where relevant, and display a clear result.

Assumptions

Assume required validity is 28 days. Plan data: (name, price, included GB, validity, extra-GB rate).

Algorithm

1. Start. 2. Read the required inputs. 3. Validate inputs. 4. Calculate totals and apply the specified conditions. 5. Display the result. 6. Stop.

Flowchart (text form)

START → Input → Validate → Calculate / Apply Conditions → Display Result → STOP

Pseudocode

```
START\nINPUT required values\nVALIDATE input values\nCALCULATE totals and apply conditions\nDISPLAY result\nSTOP
```

Python Source Code

```
usage = 12
budget = 500
plans = [
    ("Basic", 249, 8, 28, 20),
    ("Smart", 399, 15, 56, 15),
    ("Max", 499, 25, 56, 10)
]
eligible = []
for name, price, included, days, extra_rate in plans:
    cost = price + max(0, usage - included) * extra_rate
    print(name, "cost Rs.", cost)
    if days >= 28 and cost <= budget:
        eligible.append((cost, name))
if eligible:
    print("Best plan:", min(eligible)[1])
else:
    print("No eligible plan")
```

Sample Test Case / Expected Result

Expected: Basic costs Rs. 329 and is the cheapest eligible plan.

Conclusion

The program uses beginner-level Python variables, arithmetic, conditions, loops, and lists to solve the case study under the stated assumptions.

Scanned with OKEN Scanner

6. Railway Fare and Reservation Calculator

Problem Statement

Calculate fares for passengers using distance, class, age concession, and cancellation penalty.

Objectives

Apply conditions and calculations correctly, validate inputs where relevant, and display a clear result.

Assumptions

Assume Sleeper Rs. 1/km, AC Rs. 2.5/km, reservation Rs. 40; under 12 get 50% fare and age 60+ get 20% fare concession. Cancellation penalty is 25% of discounted fare plus reservation.

Algorithm

1. Start. 2. Read the required inputs. 3. Validate inputs. 4. Calculate totals and apply the specified conditions. 5. Display the result. 6. Stop.

Flowchart (text form)

START → Input → Validate → Calculate / Apply Conditions → Display Result → STOP

Pseudocode

```
START\nINPUT required values\nVALIDATE input values\nCALCULATE totals and apply conditions\nDISPLAY result\nSTOP
```

Python Source Code

```
passengers = [
    ("Anu", 10, 300, "Sleeper", False),
    ("Ramesh", 65, 300, "AC", True)
]
rates = {"Sleeper": 1.0, "AC": 2.5}
for name, age, distance, travel_class, cancelled in passengers:
    if distance <= 0 or age < 0 or travel_class not in rates:
        print(name, "Invalid details")
        continue
    fare = distance * rates[travel_class]
    if age < 12:
        fare *= 0.50
    elif age >= 60:
        fare *= 0.80
    total = fare + 40
    print(name, "total Rs.", total)
    if cancelled:
        penalty = 0.25 * fare + 40
        print("Penalty Rs.", penalty, "Refund Rs.", max(0, total - penalty))
```

Sample Test Case / Expected Result

Expected: Anu total Rs. 190. Ramesh AC senior fare before cancellation Rs. 640.

Conclusion

The program uses beginner-level Python variables, arithmetic, conditions, loops, and lists to solve the case study under the stated assumptions.

Scanned with OKEN Scanner

7. Retail Billing with Multiple Discount Rules

Problem Statement

Create an itemized invoice with category tax, threshold discount, and a maximum discount limit.

Objectives

Apply conditions and calculations correctly, validate inputs where relevant, and display a clear result.

Assumptions

Assume groceries tax 5%, clothing 12%, electronics 18%. Discount 10% when subtotal >= Rs. 5000, capped at Rs. 1000; tax is calculated after discount.

Algorithm

1. Start. 2. Read the required inputs. 3. Validate inputs. 4. Calculate totals and apply the specified conditions. 5. Display the result. 6. Stop.

Flowchart (text form)

START → Input → Validate → Calculate / Apply Conditions → Display Result → STOP

Pseudocode

```
START\nINPUT required values\nVALIDATE input values\nCALCULATE totals and apply conditions\nDISPLAY result\nSTOP
```

Python Source Code

```
items = [
    ("Rice", 800, 2, "groceries"),
    ("Shirt", 1200, 2, "clothing"),
    ("Headphones", 2500, 1, "electronics")
]
tax_rates = {"groceries": 0.05, "clothing": 0.12, "electronics": 0.18}
subtotal = sum(price * qty for name, price, qty, cat in items)
discount = min(subtotal * 0.10, 1000) if subtotal >= 5000 else 0
share = discount / subtotal if subtotal else 0
tax_total = 0
for name, price, qty, cat in items:
    line = price * qty * (1 - share)
    tax = line * tax_rates[cat]
    tax_total += tax
    print(name, "line Rs.", price * qty, "tax Rs.", round(tax, 2))
print("Subtotal:", subtotal, "Discount:", discount)
print("Tax:", round(tax_total, 2))
print("Final amount:", round(subtotal - discount + tax_total, 2))
```

Sample Test Case / Expected Result

Expected: Subtotal Rs. 6800; discount Rs. 680; final amount = subtotal - discount + category taxes.

Conclusion

The program uses beginner-level Python variables, arithmetic, conditions, loops, and lists to solve the case study under the stated assumptions.

Scanned with OKEN Scanner

8. Manufacturing Cost and Profitability Analysis

Problem Statement

Calculate material cost, wastage, labour, machine cost, revenue, and profit target.

Objectives

Apply conditions and calculations correctly, validate inputs where relevant, and display a clear result.

Assumptions

Assume wastage is 5% of material cost and target profit is 20% of total production cost.

Algorithm

1. Start. 2. Read the required inputs. 3. Validate inputs. 4. Calculate totals and apply the specified conditions. 5. Display the result. 6. Stop.

Flowchart (text form)

START → Input → Validate → Calculate / Apply Conditions → Display Result → STOP

Pseudocode

```
START\nINPUT required values\nVALIDATE input values\nCALCULATE totals and apply conditions\nDISPLAY result\nSTOP
```

Python Source Code

```
quantity = 100
material_per_unit = 80
labour = 2500
machine = 1200
selling_price = 125
material = quantity * material_per_unit
wastage = material * 0.05
total_cost = material + wastage + labour + machine
revenue = quantity * selling_price
profit = revenue - total_cost
profit_percent = profit / total_cost * 100
print("Total cost:", total_cost)
print("Revenue:", revenue)
print("Profit:", profit)
print("Profit percentage:", round(profit_percent, 2))
print("Target met" if profit_percent >= 20 else "Target not met")
```

Sample Test Case / Expected Result

Expected: Total cost Rs. 12100; revenue Rs. 12500; profit Rs. 400; 20% target not met.

Conclusion

The program uses beginner-level Python variables, arithmetic, conditions, loops, and lists to solve the case study under the stated assumptions.

Scanned with OKEN Scanner

9. Multi-Currency Travel Budget Analysis

Problem Statement

Convert expenses to INR, include conversion fees, compare against a budget, and find the largest category.

Objectives

Apply conditions and calculations correctly, validate inputs where relevant, and display a clear result.

Assumptions

Assume supplied rates are INR per unit of foreign currency and the conversion fee is 2%.

Algorithm

1. Start. 2. Read the required inputs. 3. Validate inputs. 4. Calculate totals and apply the specified conditions. 5. Display the result. 6. Stop.

Flowchart (text form)

START → Input → Validate → Calculate / Apply Conditions → Display Result → STOP

Pseudocode

```
START\nINPUT required values\nVALIDATE input values\nCALCULATE totals and apply conditions\nDISPLAY result\nSTOP
```

Python Source Code

```
expenses = [
    ("Hotel", 200, "USD"),
    ("Food", 80, "EUR"),
    ("Transport", 50, "USD")
]
rates = {"USD": 83, "EUR": 90}
budget = 25000
totals = []
for category, amount, currency in expenses:
    converted = amount * rates[currency]
    total = converted * 1.02
    totals.append((category, total))
    print(category, "INR incl. fee:", round(total, 2))
grand_total = sum(value for category, value in totals)
print("Total:", round(grand_total, 2))
print("Largest category:", max(totals, key=lambda x: x[1])[0])
print("Budget status:", "Within budget" if grand_total <= budget else "Over budget")
```

Sample Test Case / Expected Result

Expected: Total spending Rs. 28509; largest category Hotel; budget exceeded.

Conclusion

The program uses beginner-level Python variables, arithmetic, conditions, loops, and lists to solve the case study under the stated assumptions.

Scanned with OKEN Scanner

10. Campus Event Financial Planning

Problem Statement

Calculate event costs, income, shortfall, and minimum registrations needed to break even.

Objectives

Apply conditions and calculations correctly, validate inputs where relevant, and display a clear result.

Assumptions

Assume venue, equipment, and transport are fixed costs; catering and certificates are per participant. Registration fee Rs. 500; sponsorship is fixed.

Algorithm

1. Start. 2. Read the required inputs. 3. Validate inputs. 4. Calculate totals and apply the specified conditions. 5. Display the result. 6. Stop.

Flowchart (text form)

START → Input → Validate → Calculate / Apply Conditions → Display Result → STOP

Pseudocode

```
START\nINPUT required values\nVALIDATE input values\nCALCULATE totals and apply conditions\nDISPLAY result\nSTOP
```

Python Source Code

```
venue = 12000
equipment = 5000
transport = 3000
catering = 250
certificate = 50
sponsorship = 8000
fee = 500
participants = 100
fixed = venue + equipment + transport
variable = catering + certificate
total_cost = fixed + variable * participants
income = sponsorship + fee * participants
shortfall = max(0, total_cost - income)
net_per_person = fee - variable
needed = max(0, (fixed - sponsorship + net_per_person - 1) // net_per_person)
print("Total event cost:", total_cost)
print("Total income:", income)
print("Shortfall:", shortfall)
print("Minimum registrations to break even:", needed)
```

Sample Test Case / Expected Result

Expected: Total cost Rs. 50000; income Rs. 58000; shortfall Rs. 0; break-even registrations 30.

Conclusion

The program uses beginner-level Python variables, arithmetic, conditions, loops, and lists to solve the case study under the stated assumptions.

Scanned with OKEN Scanner